deti universidade de aveiro
departamento de electrónica, telecomunicações e informática

HW1: Mid-term assignment report

Software Testing and Quality Control

Duarte Gabriel Castro Branco - 119253

2024/2025

Conteúdo

1 Introduction **1**
1.1 Overview of the work 1
1.2 Current implementation (faults & extras) 1

2 Product specification **2**
2.1 Functional scope and supported interactions 2
2.2 System implementation architecture 3
2.3 API for developers 4

3 Quality assurance **6**
3.1 Overall strategy for testing 6
3.2 Unit and integration testing 6
3.3 Acceptance testing 7
3.4 Non-functional testing 7
3.5 Code quality analysis 7
3.6 Continuous integration pipeline 8

4 References & resources **10**

1. Introduction

1.1 Overview of the work

This report presents the midterm individual project developed for the *Teste e Qualidade de Software* (TQS) course. This project, `ZeroMonos`, is a simple full-stack web app designed for a fictional waste management company that provides a garbage collection service in multiple municipalities. The system should allow citizens to book this service, by picking a date, their municipality and the type and description of the items they want to get rid of. It should also allow for staff to manage these bookings.

The system exposes a Spring Boot REST API and a `Thymeleaf` web app for the front-end. It supports both citizen and staff facing interactions and integrates a complete automated testing strategy (unit, service, integration, function, performance and code quality analysis).

1.2 Current implementation (faults & extras)

All core function requirements from the assignment were implemented:

- Citizens can create bookings specifying their name, municipality, date and item description.
- Municipalities are dynamically fetched from an external API (https://json.geoapi.pt/municipios).
- Upon successful booking, the citizen gets a unique token with which he can check the request's state and even cancel it.
- Bookings are persisted in a `PostgreSQL` database.
- Staff can view, update and delete existing bookings, as well as browse for existing requests, per municipality.

In addition to the required functionalities, the project integrates a `Continuous Integration (CI)` pipeline using GitHub Actions. It's set up to build and test the project on each push/merge request, and it includes `SonarQube` Quality Gate enforcement via `SonarQube Cloud`.

Yet, the current implementation has some constraints and missing features such as: mobile responsiveness and authentication/login system.

2. Product specification

2.1 Functional scope and supported interactions

The `ZeroMonos` platform is designed for two primary actor types:

- **Citizens**: Regular users who want to dispose of bulky waste.
- **Staff**: Authorized personnel who oversee and manage the collection requests.

Each actor interacts with the system in different ways.
The citizen has these main interactions:

- **Create a Booking**: The citizen fills out a form with their name, selects their municipality from a list and describes the items to be collected (type and description). Upon submission, the system records the booking in the database and generates a unique token which is displayed to the user. This token acts as a key for later tracking.
- **Check Booking**: Using the token received, the citizen can navigate to a status page and input the token to retrieve the current status of their request, optionally, the user can also cancel the booking.

The staff has these main interactions:

- **Manage bookings**: The staff can view and edit the bookings created by the users. He has the ability to change the initial state of each booking (RECEIVED) to any of these: ASSIGNED (when someone has been assigned to take care of that booking), IN_PROGRESS (when the booking is being taken care right now) and COMPLETED. Also, the booking can also be completely removed.
- **Filter by Municipality**: The staff can also filter the bookings list by municipality name. This is useful if the system is full of bookings from everywhere, so the managing process is easier. The interface provides a search input where staff type the municipality (e.g. "Aveiro") and the list updates to show only matching bookings.

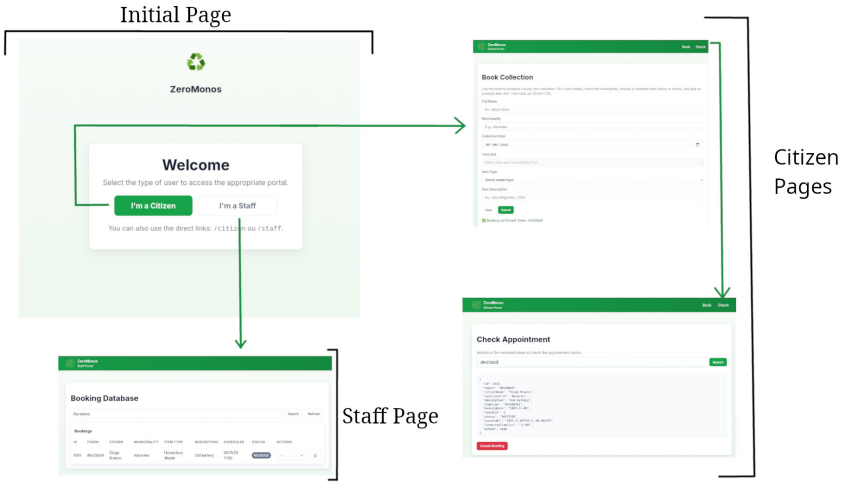

Figura 2.1: Overview of the `ZeroMonos` application flow

2.2 System implementation architecture

`ZeroMonos` follows a standard Spring Boot web application architecture with a layered structure:

- **Back-End**: The core is a Spring Boot application (Java) that exposes both `RESTful` endpoints and serves web pages. It uses a service/business layer to implement business logic (for example generating tokens, handling status changes, etc.) and a repository/persistence layer to interact with the database. Data is stored in a `PostgreSQL` database. The main data entity is the `Booking.java` with fields for ID, token, citizen name, municipality, description, status, etc.
- **Front-End**: The web front-end is built using `Thymeleaf` templates, which are served by Spring Boot controllers. The base structure and navigation are defined in `layout.html`, which provides common fragments for the header, footer, and scripts in all other pages (`index.html`, `citizen-reserve.html`, `citizen-check.html` and `staff.html`). Styling is handled by `style.css`, which defines a clean and responsive visual design. Interactivity is implemented through three JavaScript modules: `layout.js` (manages shared UI behavior), `citizen.js` (handles all requests to the API needed for the citizen-facing pages) and `staff.js` (handles API interactions for the staff-facing page).

The app also integrates with an external API for Portuguese municipalities. The application fetches the list of all municipalities by calling the `geoapi.pt` service (specifically, an endpoint providing a `JSON` list of municipalities). This is done via a Spring `RestTemplate` call in a `MunicipalityService` component. On first use, the service retrieves the list of municipality names and caches them in memory for faster usage later. Docker is also used to create the instance of a local `PostgreSQL` database.

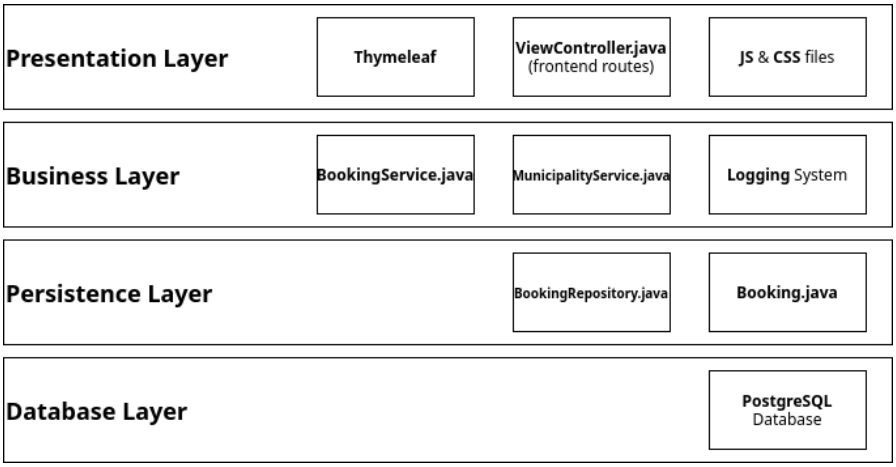

Figura 2.2: Layered architecture of `ZeroMonos`

2.3 API for developers

`ZeroMonos` provides a REST API that allows external clients to interact with the system. All endpoints use `JSON` for requests and responses. The key API endpoints include:

- **POST /bookings** - Create a new booking request. The request body should contain the necessary booking details.
- **GET /bookings/{token}** - Retrieve an existing booking by its public token. This is intended for check page to fetch the current details and status of their request.
- **GET /bookings** - Retrieve all bookings in the system. This returns a list of all Booking records (intended for staff). This endpoint also supports query parameters (e.g. GET /bookings?municipality=Aveiro&?date=2025-11-05) to filter the results by municipality and current date.
- **DELETE /bookings/{id}** - Remove a booking record (staff use). This permanently deletes the booking with the given internal ID from the database.
- **PATCH /bookings/{id}?status={newStatus}** - Update the state of a booking (staff use). The {id} in the URL is the internal ID of the booking and {newStatus} is the new state.
- **GET /bookings/available-slots** - Returns a list of available time slots for a specific date and municipality, according to the bookings already set up on that day and municioality.
- **GET /municipalities** - Gets the municipalities names from other API. It has a fallback in case the `geoapi.pt` returns nothing or something invalid.

These endpoints make it possible to interact with ZeroMonos from other front-ends or integrate it into a larger platform.

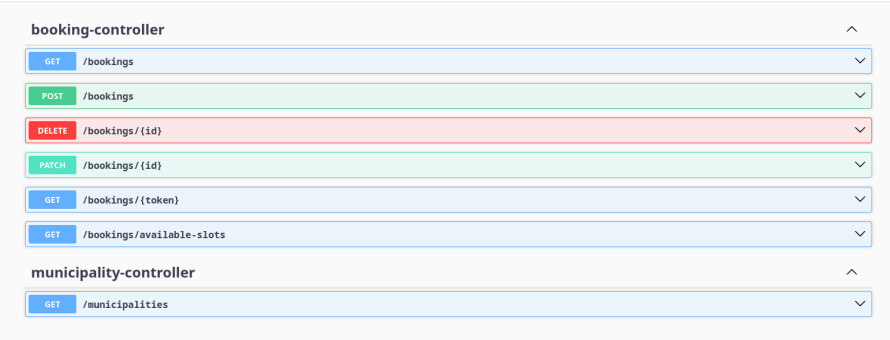

Figura 2.3: Swagger UI screenshot

3. Quality assurance

3.1 Overall strategy for testing

The project adopted a layered testing strategy combining **unit tests**, **service tests** , **integration tests**, **functional tests**, and **performance simulations**.

The tests rely on `JUnit 5` and `Mockito` for unit and service tests, and used Spring's `MockMvc` for API integration testing. `Selenium WebDriver` enabled BDD-style UI automation tests. Finally, non-functional testing was implemented using `k6` scripts to stress-test the app.

For code analysis, a CI pipeline integrated with `SonarQube Cloud` was used.

3.2 Unit and integration testing

Unit tests were used extensively for core business logic, particularly within `BookingService` and `MunicipalityService`. These tests validated **date and slot constraints**, **status transitions**, **token-based queries**, and **fallback behavior** for external APIs.

- **BookingServiceTest:** verifies key domain constraints, including:
  - A booking can only be created if the slot is free and the input is valid.
  - Status transitions (e.g., to `IN_PROGRESS` or `CANCELLED`) are correctly persisted.
  - Filtering methods work as expected for date, token, and municipality.
- **MunicipalityServiceTest:** handles the geo API and tests the fallback behavior

Integration tests leveraged Spring Boot's `@SpringBootTest` and *MockMvc* to test real HTTP interactions and application state via the controllers. Scenarios covered include all the main REST operations from the defined API (see Section 2.3).

- **BookingControllerTest** validates all booking related API flows.
- **MunicipalityControllerTest** validates the return of the municipalities list.

Additionally, domain-level validation was tested in `BookingValidationTest`, ensuring that entity constraints on fields like name, description, item type, and time slot conform to validation rules. These tests catch input issues before persistence or logic execution.

3.3 Acceptance testing

The end-user functionality was tested using Selenium WebDriver in a BDD-like style. Firefox was used in headless mode for full-stack tests involving the booking and status-checking UI. These functional tests (`FunctionalSeleniumTest`) are written using the Page Object Model (`BookReservePage`, `CitizenCheckPage`, `StaffPage`) and cover the following workflows:

- Citizen successfully books two collection slots in different municipalities.
- Staff views that newly created bookings and updates it's state.
- Staff filters the bookings by municipality, seeing them both separated.
- Citizen checks booking status using a token and cancels it.

Tests were executed sequentially, reusing state between steps when necessary, and included verification of UI changes and database effects.

I also generated a `Lighthouse` report for the intial web page and got very high scores (see Fig 3.1).

3.4 Non-functional testing

Performance and load testing was carried out using `k6`. Three scenarios were tested against the `POST /bookings` endpoint:

- **Baseline load** (`performance.js`): simulates standard traffic to verify stable response times and success rate.
- **Stress test** (`stress-test.js`): gradually increases user load to determine system limits and monitor degradation.
- **Spike test** (`spike-test.js`): applies sudden load surges to assess system recovery and responsiveness.

All tests passed and they validated the back-end's ability to handle booking requests under varying load profiles without performance breakdown.

3.5 Code quality analysis

Static code analysis was performed using `SonarQube`, integrated into the development pipeline via GitHub Actions. The project was analyzed on `SonarCloud`, where each push or pull request triggers automatic scanning and *Quality Gate* reinforcement.

The configured *Quality Gate* enforces a minimum coverage threshold and blocks merges if critical issues, bugs, or major code smells are introduced (typical Sonar Way).

The project passed the configured *Quality Gate*, but initial scans revealed several issues that were later resolved such as:

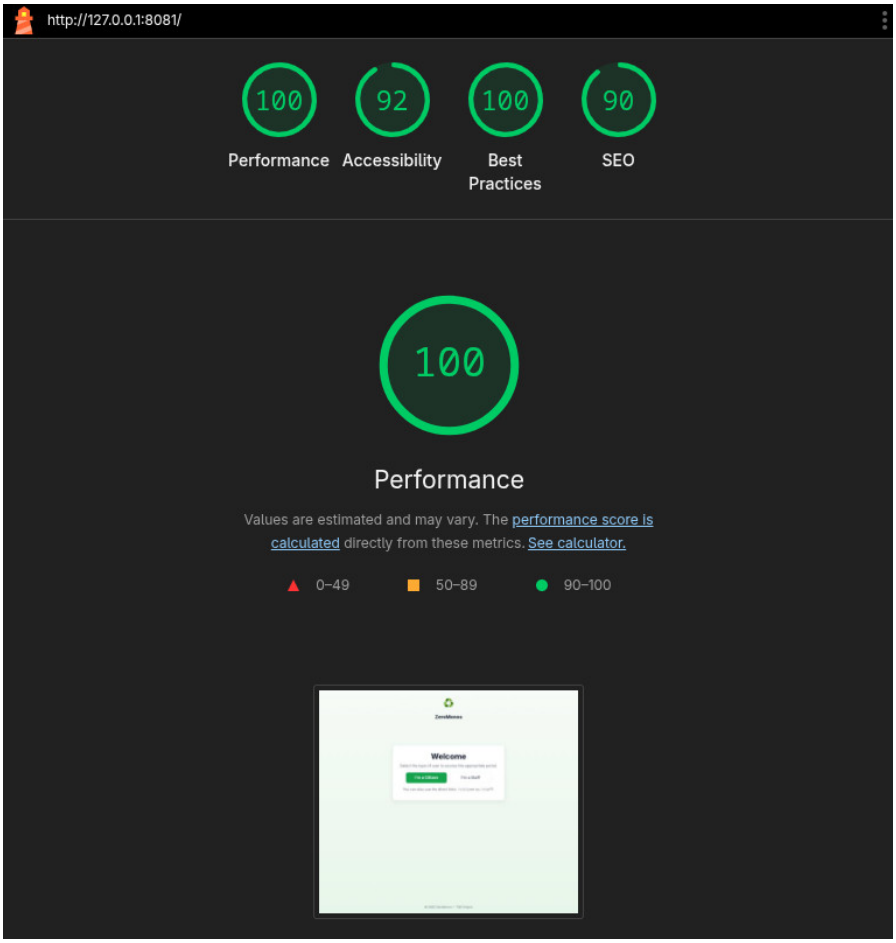

Figura 3.1: Lighthouse report

- **Security**: The use of `@CrossOrigin` without restriction allowed all domains by default. This was flagged as a security and I needed to manually allow traffic from my app's controllers.
- **Maintainability**: There were 2 main issues. First the use of hardcodeded pauses (`Thread.sleep()`) on tests, which were replaced with condition-based waits. Second, there some logging being done via `System.err.println`, which were replaced with SLF4J logging to allow proper log levels and structured output.

After these issues were solved, the `SonarQube` report showed no issues at all and total code coverage of 88.3%, which is in the optimal threshold of >= 80% (see Fig 3.2).

3.6 Continuous integration pipeline

A full continuous integration (CI) workflow was implemented using **GitHub Actions**, defined in `ci.yml`. The pipeline runs automatically on every push and pull request to

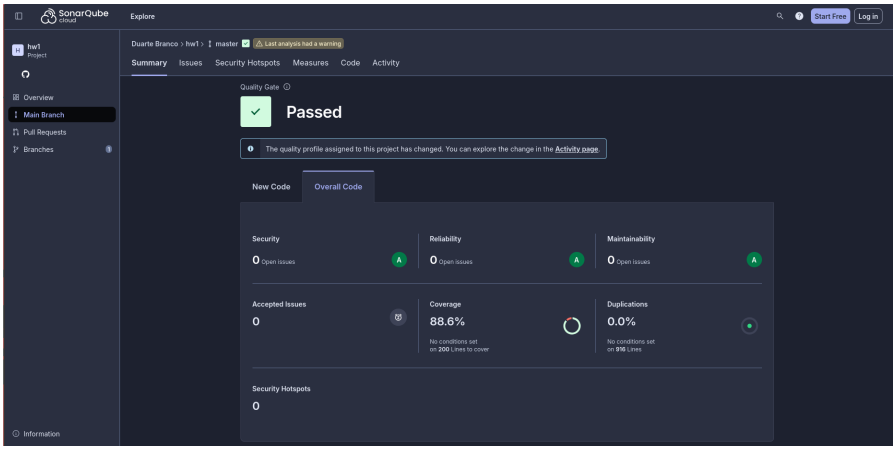

Figura 3.2: `SonarQube` dashboard

the repository, enforcing build integrity, test execution, and code quality checks.

The CI workflow includes the following stages:

1. **Environment Setup:** A `PostgreSQL` service is launched in Docker using GitHub-hosted runners. The Spring Boot application connects to this database during test execution.

2. **Readiness check:** A manual loop using `nc -z localhost 5432` ensures PostgreSQL is reachable before proceeding with the build.

3. **Build and static analysis:** The Maven build uses `clean verify` to compile and test the project. Immediately afterward, the `sonar-maven-plugin` uploads coverage and static analysis metrics to **SonarCloud**.

4. **Quality Gate enforcement:** The job waits for the SonarCloud Quality Gate result using the `sonarsource/sonarqube-quality-gate-action`. If the gate is not passed (e.g., low coverage, code smells, or vulnerabilities), the workflow fails and the pull request is blocked.

This pipeline ensures that only clean, tested, and maintainable code is merged into the main branch. It also provides fast feedback to developers by catching regressions and enforcing standards at every commit.

4. References & resources

- **Video demo**: `/docs/demo.mp4`
- **Public Repository** (for *SonarQube Cloud* to work)
- **Quality Gate dashboard**